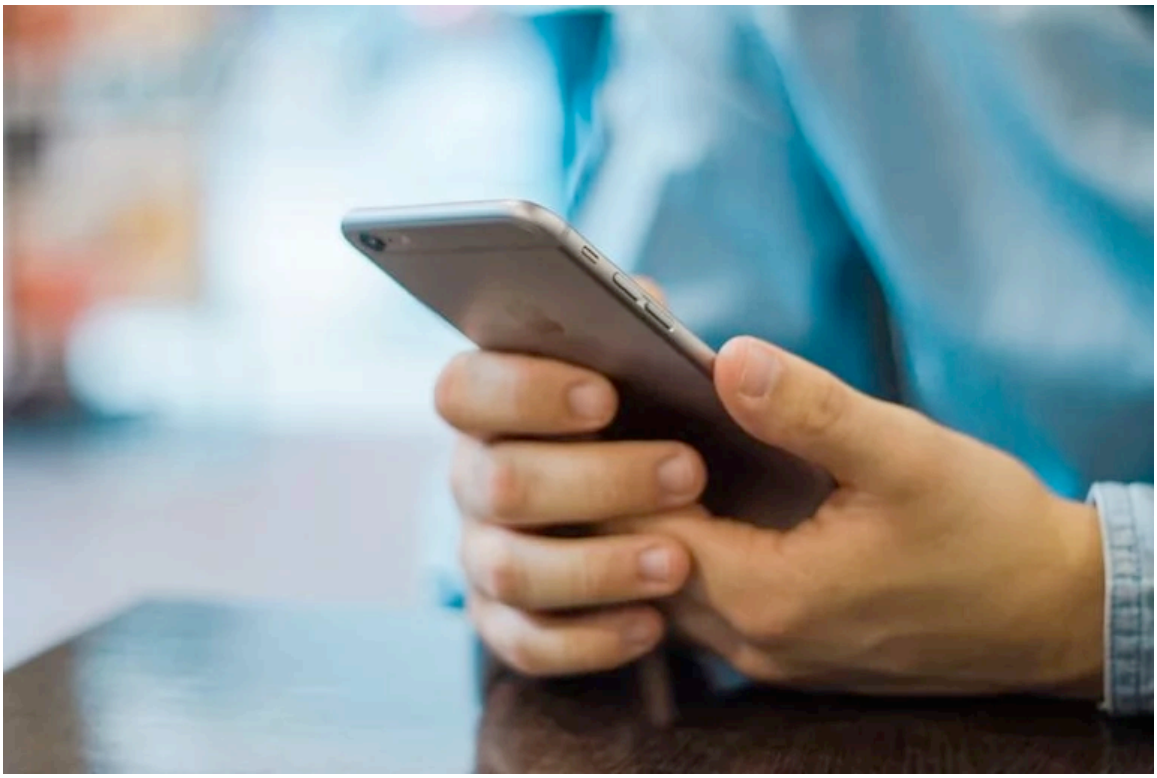
Árvores II - Tarefa I



fonte (https://momentum.org/programs-services/manage-your-money/savings-app/pxfuel-creative-commons-zero-cc0-iphone-smartphone-cell-phone-mobile-technology-texting/)

Neste exercício vamos trabalhar com a base de dados de identificação de atividade humana com smartphones. Smartphones possuem acelerômetro e giroscópio, que registram dados de aceleração e giro nos eixos X, Y e Z, com frequencia de 50 Hz (ou seja, 50 registros por segundo). Os dados foram agrupados a medidas de 0.3 Hz, calculando-se variáveis derivadas como aceleração mínima, máxima, média etc por eixo no período agrupado de tempo, resultando em 561 variáveis que podem ser lidas nas bases disponíveis.

A base é oriunda de um experimento, em que os indivíduos realizavam uma de seis atividades corriqueiras:

- andando
- subindo escada
- descendo escada
- parado
- sentado
- deitado

O objetivo é classificar a atividade humana com base nos dados do acelerômetro e giroscópio do celular.

1. Carregar a base

Sua primeira atividade é carregar a base.

Ela está disponível neste link:
https://archive.ics.uci.edu/ml/datasets/human+activity+recognition+using+smartphones (https://archive.ics.uci.edu/ml/datasets/human+activity+recognition+using+smartphones)

dados https://archive.ics.uci.edu/ml/machine-learning-databases/00240/ (https://archive.ics.uci.edu/ml/machine-learning-databases/00240/)
dataset.names https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR%20Dataset.names (https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR%20Dataset.names)
UCI HAR Dataset.zip https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR%20Dataset.zip (https://archive.ics.uci.edu/ml/machine-learning-databases/00240/UCI%20HAR%20Dataset.zip)

Você vai encontrar os seguintes arquivos:

- **features.txt:** contém a lista com os nomes das variáveis
- **features_info.txt:** contém a descrição das variáveis
- **README.txt:** contém uma descrição do estudo e das bases
- **activity_labels:** contém o código da atividade (inteiro entre 1 e 6) e a descrição
- **subject_train.txt:** uma lista indicando que registro pertence a que indivíduo na base de treino
- **X_train.txt:** as *features* (ou variáveis explicativas) da base de testes. Cada linha representa um registro das informações de um indivíduo em um intervalo de tempo de aproximadamente 1/0.3 segundo. As medidas estão em ordem temporal dentro do estudo, e o indivíduo que originou a medida está identificado na base subject_train.txt.
- **y_train.txt:** Possui o mesmo número de linhas que X_test. Contém um número de 1 a 6 indicando a atividade que estava sendo realizada por aquele registro na base de treino.
- **subject_test.txt:** uma lista indicando que registro pertence a que indivíduo na base de teste
- **X_test.txt:** as *features* (ou variáveis explicativas) da base de testes. Cada linha representa um registro das informações de um indivíduo em um intervalo de tempo de aproximadamente 1/0.3 segundo. As medidas estão em ordem temporal dentro do estudo, e o indivíduo que originou a medida está identificado na base subject_test.txt.
- **y_test.txt:** Possui o mesmo número de linhas que X_train. Contém um número de 1 a 6 indicando a atividade que estava sendo realizada por aquele registro na base de teste.

Carregue as bases:

1. Faça o download dos arquivos.
2. Carregue a base `features.txt` em uma *Series* (por exemplo usando o comando `pd.read_csv()`.
3. Carregue a base subject_train.txt em uma *Series*
4. Carregue a base X_train.txt
   A. Faça com que as colunas deste *dataframe* tenham os nomes indicados em `features.txt`
   B. Sem alterar a ordem dos *dataframes*, coloque o indicador do indivíduo lido em `subject_train.txt` como uma variável a mais neste *dataframe***
   C. Faça com que este *dataframe* tenha um índice duplo, composto pela ordem dos dados e pelo identificador do indivíduo
5. Com a mesma lógica, carregue a base X_test
   A. Certifique-se de que tenha os nomes lidos em `features.txt`
   B. Coloque o identificador do sujeito lido em `subject_test.txt`

C. Defina um índice duplo composto pela ordem do registro e o identificador do registro

6. Salve as bases em arquivos CSV para facilitar a leitura deles na terefa 2 deste módulo
7. Considere que esta base é maior que a da aula, tanto em linhas quanto em colunas. Selecione apenas as três primeiras colunas da base ('tBodyAcc-mean()-X', 'tBodyAcc-mean()-Y' e 'tBodyAcc-mean()-Z'), para efeitos desse exercício.

import pandas as pd from sklearn.tree import DecisionTreeClassifier import numpy as np import matplotlib.pyplot as plt

In [ ]:

In [ ]:

In [66]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

Os arquivos da base UCI HAR Dataset já foram baixados e estão disponíveis na pasta do projeto.

In [ ]:

2

In [67]:
```
features = pd.read_csv(
    './UCI HAR Dataset/features.txt',
    sep=r'\s+',
    header=None,
    names=['index', 'feature_name']
)

features_series = features['feature_name']

print("Quantidade de features:", len(features_series))
print(features_series.head())
```

```
Quantidade de features: 561
0    tBodyAcc-mean()-X
1    tBodyAcc-mean()-Y
2    tBodyAcc-mean()-Z
3     tBodyAcc-std()-X
4     tBodyAcc-std()-Y
Name: feature_name, dtype: object
```

In [ ]:

3

In [68]:
```
subject_train = pd.read_csv(
    './UCI HAR Dataset/train/subject_train.txt',
    header=None,
    names=['subject']
)['subject']

print("Quantidade de registros em subject_train:", len(subject_train))
print(subject_train.head())
```

```
Quantidade de registros em subject_train: 7352
0    1
1    1
2    1
3    1
4    1
Name: subject, dtype: int64
```

In [ ]:

4A

In [69]:
```
X_train = pd.read_csv(
    './UCI HAR Dataset/train/X_train.txt',
    sep=r'\s+',
    header=None
)

X_train.columns = features_series

print("Dimensão de X_train:", X_train.shape)
print(X_train.head())
```

```
Dimensão de X_train: (7352, 561)
feature_name  tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  \
0                      0.288585          -0.020294          -0.132905
1                      0.278419          -0.016411          -0.123520
2                      0.279653          -0.019467          -0.113462
3                      0.279174          -0.026201          -0.123283
4                      0.276629          -0.016570          -0.115362

feature_name  tBodyAcc-std()-X  tBodyAcc-std()-Y  tBodyAcc-std()-Z  \
0                    -0.995279         -0.983111         -0.913526
1                    -0.998245         -0.975300         -0.960322
2                    -0.995380         -0.967187         -0.978944
3                    -0.996091         -0.983403         -0.990675
4                    -0.998139         -0.980817         -0.990482

feature_name  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  tBodyAcc-mad()-Z  \
0                    -0.995112         -0.983185         -0.923527
1                    -0.998807         -0.974914         -0.957686
2                    -0.996520         -0.963668         -0.977469
3                    -0.997099         -0.982750         -0.989302
4                    -0.998321         -0.979672         -0.990441

feature_name  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-meanFreq()  \
0                    -0.934724  ...                        -0.074323
1                    -0.943068  ...                         0.158075
2                    -0.938692  ...                         0.414503
3                    -0.938692  ...                         0.404573
4                    -0.942469  ...                         0.087753

feature_name  fBodyBodyGyroJerkMag-skewness()  \
0                                   -0.298676
1                                   -0.595051
2                                   -0.390748
3                                   -0.117290
4                                   -0.351471

feature_name  fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravity)
\
0                                   -0.710304                    -0.112754
1                                   -0.861499                     0.053477
2                                   -0.760104                    -0.118559
3                                   -0.482845                    -0.036788
4                                   -0.699205                     0.123320

feature_name  angle(tBodyAccJerkMean),gravityMean)  \
0                                          0.030400
1                                         -0.007435
2                                          0.177899
3                                         -0.012892
4                                          0.122542

feature_name  angle(tBodyGyroMean,gravityMean)  \
0                                    -0.464761
1                                    -0.732626
2                                     0.100699
3                                     0.640011
4                                     0.693578

feature_name  angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)
\
0                                         -0.018446             -0.841247
```

```
1                                          0.703511             -0.844788
2                                          0.808529             -0.848933
3                                         -0.485366             -0.848649
4                                         -0.615971             -0.847865

feature_name  angle(Y,gravityMean)  angle(Z,gravityMean)
0                         0.179941             -0.058627
1                         0.180289             -0.054317
2                         0.180637             -0.049118
3                         0.181935             -0.047663
4                         0.185151             -0.043892

[5 rows x 561 columns]
```

In [ ]:

```

```

4B

In [70]:

```
X_train['subject'] = subject_train.values

print(X_train[['subject']].head())
```

```
feature_name  subject
0                   1
1                   1
2                   1
3                   1
4                   1
```

In [ ]:

```

```

4C criar indice duplo

In [71]:
```
X_train['ordem'] = range(X_train.shape[0])

X_train.set_index(['ordem', 'subject'], inplace=True)

print(X_train.head())
print(X_train.index)
```

```
feature_name   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  \
ordem subject
0     1                 0.288585          -0.020294          -0.132905
1     1                 0.278419          -0.016411          -0.123520
2     1                 0.279653          -0.019467          -0.113462
3     1                 0.279174          -0.026201          -0.123283
4     1                 0.276629          -0.016570          -0.115362

feature_name   tBodyAcc-std()-X  tBodyAcc-std()-Y  tBodyAcc-std()-Z  \
ordem subject
0     1               -0.995279         -0.983111         -0.913526
1     1               -0.998245         -0.975300         -0.960322
2     1               -0.995380         -0.967187         -0.978944
3     1               -0.996091         -0.983403         -0.990675
4     1               -0.998139         -0.980817         -0.990482

feature_name   tBodyAcc-mad()-X  tBodyAcc-mad()-Y  tBodyAcc-mad()-Z  \
ordem subject
0     1               -0.995112         -0.983185         -0.923527
1     1               -0.998807         -0.974914         -0.957686
2     1               -0.996520         -0.963668         -0.977469
3     1               -0.997099         -0.982750         -0.989302
4     1               -0.998321         -0.979672         -0.990441

feature_name   tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-meanFreq()  \
ordem subject                    ...
0     1               -0.934724  ...                        -0.074323
1     1               -0.943068  ...                         0.158075
2     1               -0.938692  ...                         0.414503
3     1               -0.938692  ...                         0.404573
4     1               -0.942469  ...                         0.087753

feature_name   fBodyBodyGyroJerkMag-skewness()  \
ordem subject
0     1                              -0.298676
1     1                              -0.595051
2     1                              -0.390748
3     1                              -0.117290
4     1                              -0.351471

feature_name   fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravit
y)  \
ordem subject
0     1                              -0.710304                   -0.11275
4
1     1                              -0.861499                    0.05347
7
2     1                              -0.760104                   -0.11855
9
3     1                              -0.482845                   -0.03678
8
4     1                              -0.699205                    0.12332
0

feature_name   angle(tBodyAccJerkMean),gravityMean)  \
ordem subject
0     1                                   0.030400
1     1                                  -0.007435
2     1                                   0.177899
3     1                                  -0.012892
4     1                                   0.122542
```

```
feature_name   angle(tBodyGyroMean,gravityMean)  \
ordem subject
0     1                                -0.464761
1     1                                -0.732626
2     1                                 0.100699
3     1                                 0.640011
4     1                                 0.693578

feature_name   angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)
\
ordem subject
0     1                                   -0.018446             -0.841247
1     1                                    0.703511             -0.844788
2     1                                    0.808529             -0.848933
3     1                                   -0.485366             -0.848649
4     1                                   -0.615971             -0.847865

feature_name   angle(Y,gravityMean)  angle(Z,gravityMean)
ordem subject
0     1                    0.179941             -0.058627
1     1                    0.180289             -0.054317
2     1                    0.180637             -0.049118
3     1                    0.181935             -0.047663
4     1                    0.185151             -0.043892

[5 rows x 561 columns]
MultiIndex([(   0,  1),
            (   1,  1),
            (   2,  1),
            (   3,  1),
            (   4,  1),
            (   5,  1),
            (   6,  1),
            (   7,  1),
            (   8,  1),
            (   9,  1),
            ...
            (7342, 30),
            (7343, 30),
            (7344, 30),
            (7345, 30),
            (7346, 30),
            (7347, 30),
            (7348, 30),
            (7349, 30),
            (7350, 30),
            (7351, 30)],
           names=['ordem', 'subject'], length=7352)
```

In [ ]:

5.

In [72]:
```
subject_test = pd.read_csv(
    './UCI HAR Dataset/test/subject_test.txt',
    header=None,
    names=['subject']
)['subject']

print("Quantidade de registros em subject_test:", len(subject_test))
print(subject_test.head())
```

```
Quantidade de registros em subject_test: 2947
0    2
1    2
2    2
3    2
4    2
Name: subject, dtype: int64
```

In [ ]:

5.A Ler X_test e nomear colunas

In [73]:
```
X_test = pd.read_csv(
    './UCI HAR Dataset/test/X_test.txt',
    sep=r'\s+',
    header=None
)

X_test.columns = features_series

print("Dimensão de X_test:", X_test.shape)
print(X_test.head())
```

```
Dimensão de X_test: (2947, 561)
feature_name  tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  \
0                      0.257178          -0.023285          -0.014654
1                      0.286027          -0.013163          -0.119083
2                      0.275485          -0.026050          -0.118152
3                      0.270298          -0.032614          -0.117520
4                      0.274833          -0.027848          -0.129527

feature_name  tBodyAcc-std()-X  tBodyAcc-std()-Y  tBodyAcc-std()-Z  \
0                    -0.938404         -0.920091         -0.667683
1                    -0.975415         -0.967458         -0.944958
2                    -0.993819         -0.969926         -0.962748
3                    -0.994743         -0.973268         -0.967091
4                    -0.993852         -0.967445         -0.978295

feature_name  tBodyAcc-mad()-X  tBodyAcc-mad()-Y  tBodyAcc-mad()-Z  \
0                    -0.952501         -0.925249         -0.674302
1                    -0.986799         -0.968401         -0.945823
2                    -0.994403         -0.970735         -0.963483
3                    -0.995274         -0.974471         -0.968897
4                    -0.994111         -0.965953         -0.977346

feature_name  tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-meanFreq()  \
0                    -0.894088  ...                          0.071645
1                    -0.894088  ...                         -0.401189
2                    -0.939260  ...                          0.062891
3                    -0.938610  ...                          0.116695
4                    -0.938610  ...                         -0.121711

feature_name  fBodyBodyGyroJerkMag-skewness()  \
0                                   -0.330370
1                                   -0.121845
2                                   -0.190422
3                                   -0.344418
4                                   -0.534685

feature_name  fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravity)
\
0                                   -0.705974                     0.006462
1                                   -0.594944                    -0.083495
2                                   -0.640736                    -0.034956
3                                   -0.736124                    -0.017067
4                                   -0.846595                    -0.002223

feature_name  angle(tBodyAccJerkMean),gravityMean)  \
0                                          0.162920
1                                          0.017500
2                                          0.202302
3                                          0.154438
4                                         -0.040046

feature_name  angle(tBodyGyroMean,gravityMean)  \
0                                    -0.825886
1                                    -0.434375
2                                     0.064103
3                                     0.340134
4                                     0.736715

feature_name  angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)
\
0                                          0.271151             -0.720009
```

```
1                                          0.920593             -0.698091
2                                          0.145068             -0.702771
3                                          0.296407             -0.698954
4                                         -0.118545             -0.692245

feature_name  angle(Y,gravityMean)  angle(Z,gravityMean)
0                         0.276801             -0.057978
1                         0.281343             -0.083898
2                         0.280083             -0.079346
3                         0.284114             -0.077108
4                         0.290722             -0.073857

[5 rows x 561 columns]
```

In [ ]:

5B Adicionar o sujeito

In [74]:

```
X_test['subject'] = subject_test.values

print(X_test[['subject']].head())
```

```
feature_name  subject
0                   2
1                   2
2                   2
3                   2
4                   2
```

In [ ]:

5C

In [75]:
```
X_test['ordem'] = range(X_test.shape[0])

X_test.set_index(['ordem', 'subject'], inplace=True)

print(X_test.head())
print(X_test.index)
```

```
feature_name   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z  \
ordem subject
0     2                 0.257178          -0.023285          -0.014654
1     2                 0.286027          -0.013163          -0.119083
2     2                 0.275485          -0.026050          -0.118152
3     2                 0.270298          -0.032614          -0.117520
4     2                 0.274833          -0.027848          -0.129527

feature_name   tBodyAcc-std()-X  tBodyAcc-std()-Y  tBodyAcc-std()-Z  \
ordem subject
0     2               -0.938404         -0.920091         -0.667683
1     2               -0.975415         -0.967458         -0.944958
2     2               -0.993819         -0.969926         -0.962748
3     2               -0.994743         -0.973268         -0.967091
4     2               -0.993852         -0.967445         -0.978295

feature_name   tBodyAcc-mad()-X  tBodyAcc-mad()-Y  tBodyAcc-mad()-Z  \
ordem subject
0     2               -0.952501         -0.925249         -0.674302
1     2               -0.986799         -0.968401         -0.945823
2     2               -0.994403         -0.970735         -0.963483
3     2               -0.995274         -0.974471         -0.968897
4     2               -0.994111         -0.965953         -0.977346

feature_name   tBodyAcc-max()-X  ...  fBodyBodyGyroJerkMag-meanFreq()  \
ordem subject                    ...
0     2               -0.894088  ...                          0.071645
1     2               -0.894088  ...                         -0.401189
2     2               -0.939260  ...                          0.062891
3     2               -0.938610  ...                          0.116695
4     2               -0.938610  ...                         -0.121711

feature_name   fBodyBodyGyroJerkMag-skewness()  \
ordem subject
0     2                              -0.330370
1     2                              -0.121845
2     2                              -0.190422
3     2                              -0.344418
4     2                              -0.534685

feature_name   fBodyBodyGyroJerkMag-kurtosis()  angle(tBodyAccMean,gravit
y)  \
ordem subject
0     2                              -0.705974                    0.00646
2
1     2                              -0.594944                   -0.08349
5
2     2                              -0.640736                   -0.03495
6
3     2                              -0.736124                   -0.01706
7
4     2                              -0.846595                   -0.00222
3

feature_name   angle(tBodyAccJerkMean),gravityMean)  \
ordem subject
0     2                                    0.162920
1     2                                    0.017500
2     2                                    0.202302
3     2                                    0.154438
4     2                                   -0.040046
```

```
feature_name   angle(tBodyGyroMean,gravityMean)  \
ordem subject
0     2                                -0.825886
1     2                                -0.434375
2     2                                 0.064103
3     2                                 0.340134
4     2                                 0.736715

feature_name   angle(tBodyGyroJerkMean,gravityMean)  angle(X,gravityMean)
\
ordem subject
0     2                                     0.271151             -0.720009
1     2                                     0.920593             -0.698091
2     2                                     0.145068             -0.702771
3     2                                     0.296407             -0.698954
4     2                                    -0.118545             -0.692245

feature_name   angle(Y,gravityMean)  angle(Z,gravityMean)
ordem subject
0     2                    0.276801             -0.057978
1     2                    0.281343             -0.083898
2     2                    0.280083             -0.079346
3     2                    0.284114             -0.077108
4     2                    0.290722             -0.073857

[5 rows x 561 columns]
MultiIndex([(   0,  2),
            (   1,  2),
            (   2,  2),
            (   3,  2),
            (   4,  2),
            (   5,  2),
            (   6,  2),
            (   7,  2),
            (   8,  2),
            (   9,  2),
            ...
            (2937, 24),
            (2938, 24),
            (2939, 24),
            (2940, 24),
            (2941, 24),
            (2942, 24),
            (2943, 24),
            (2944, 24),
            (2945, 24),
            (2946, 24)],
           names=['ordem', 'subject'], length=2947)
```

In [ ]:

```

```

Para treinar a árvore

In [76]:
```
y_train = pd.read_csv(
    './UCI HAR Dataset/train/y_train.txt',
    header=None,
    names=['activity']
)['activity']

y_test = pd.read_csv(
    './UCI HAR Dataset/test/y_test.txt',
    header=None,
    names=['activity']
)['activity']

print("y_train:")
print(y_train.head())

print("\ny_test:")
print(y_test.head())
```

```
y_train:
0    5
1    5
2    5
3    5
4    5
Name: activity, dtype: int64

y_test:
0    5
1    5
2    5
3    5
4    5
Name: activity, dtype: int64
```

In [ ]:

6

In [77]:
```
X_train.to_csv('X_train.csv')
X_test.to_csv('X_test.csv')
y_train.to_csv('y_train.csv', index=False)
y_test.to_csv('y_test.csv', index=False)

print("Arquivos CSV salvos com sucesso.")
```

```
Arquivos CSV salvos com sucesso.
```

In [ ]:

7

In [78]:
```
colunas_selecionadas = [
    'tBodyAcc-mean()-X',
    'tBodyAcc-mean()-Y',
    'tBodyAcc-mean()-Z'
]

X_train_3 = X_train[colunas_selecionadas].copy()
X_test_3 = X_test[colunas_selecionadas].copy()

print("X_train_3:")
print(X_train_3.head())

print("\nX_test_3:")
print(X_test_3.head())
```

```
X_train_3:
feature_name   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z
ordem subject
0     1                 0.288585          -0.020294          -0.132905
1     1                 0.278419          -0.016411          -0.123520
2     1                 0.279653          -0.019467          -0.113462
3     1                 0.279174          -0.026201          -0.123283
4     1                 0.276629          -0.016570          -0.115362

X_test_3:
feature_name   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z
ordem subject
0     2                 0.257178          -0.023285          -0.014654
1     2                 0.286027          -0.013163          -0.119083
2     2                 0.275485          -0.026050          -0.118152
3     2                 0.270298          -0.032614          -0.117520
4     2                 0.274833          -0.027848          -0.129527
```

In [ ]:

2. Ajustar uma árvore de decisão

- 2.1 ajuste uma árvore de decisão com número mínimo de observações por folha = 20. Certifique-se de que você esteja utilizando apenas as 3 variáveis indicadas no exercício anterior.
- 2.2 Calcule os ccp_alphas. Como feito em aula, certifique-se de que todos os valores são positivos, e selecione apenas valores únicos.
- 2.3 Considere que vamos fazer uma árvore para cada valor de `ccp_alpha` . Para ter um pouco mais de velocidade, crie uma coleção de dados com os `ccp_alphas` , considerando apenas 1 a cada 5 valores. Dica: utilize o slicing do tipo `array[::5]` para isto. Caso se sinta seguro, fique à vontade para utilizar mais valores de `ccp_alpha` .

In [ ]:

2.1

In [79]:
```
clf = DecisionTreeClassifier(
    random_state=100,
    min_samples_leaf=20
)

clf.fit(X_train_3, y_train)

print("Árvore treinada com sucesso.")
print("Acurácia treino:", clf.score(X_train_3, y_train))
print("Acurácia teste:", clf.score(X_test_3, y_test))
```

```
Árvore treinada com sucesso.
Acurácia treino: 0.5639281828073993
Acurácia teste: 0.4407872412623006
```

In [ ]:

2.2.

In [80]:
```
path = clf.cost_complexity_pruning_path(X_train_3, y_train)
ccp_alphas = path.ccp_alphas

ccp_alphas = ccp_alphas[ccp_alphas >= 0]
ccp_alphas = np.unique(ccp_alphas)

print("Quantidade de ccp_alphas únicos e não negativos:", len(ccp_alphas))
print(ccp_alphas[:10])
```

```
Quantidade de ccp_alphas únicos e não negativos: 233
[0.00000000e+00 6.96674540e-06 1.12766571e-05 2.02554065e-05
 2.72034820e-05 2.84988860e-05 4.63397246e-05 5.35004552e-05
 5.84401136e-05 6.17808017e-05]
```

In [ ]:

2.3.

In [81]:
```
ccp_alphas_sub = ccp_alphas[::5]

print("Quantidade de ccp_alphas selecionados:", len(ccp_alphas_sub))
print(ccp_alphas_sub[:10])
```

```
Quantidade de ccp_alphas selecionados: 47
[0.00000000e+00 2.84988860e-05 6.59866112e-05 8.01474017e-05
 9.63598696e-05 1.10109332e-04 1.36017410e-04 1.49492770e-04
 1.60628990e-04 1.75578030e-04]
```

In [ ]:

In [82]:
```
%%time
```

```
UsageError: %%time is a cell magic, but the cell body is empty. Did you me
an the line magic %time (single %)?
```

3. Desempenho da árvore por ccp_alpha

- 3.1: Rode uma árvore para cada `ccp_alpha` , salvando cada árvore em uma lista
- 3.2: Calcule a acurácia de cada árvore na base de treinamento e de teste
- 3.3: Monte um gráfico da acurácia em função do `ccp_alpha` na base de validação e na base de teste
- 3.4: Selecione a melhor árvore como sendo aquela que possui melhor acurácia na base de teste
- 3.5: Qual a melhor acurácia que se pode obter com estas três variáveis?

Dica: utilize a estrutura do notebook apresentado em aula.
Dica 2: meça o tempo com a função mágica `%%time` na primeira linha da célula.
Sugestão: caso fique confortável com o tempo de execução, faça a busca pelo melhor `ccp_alpha` com mais iterações.
Sugestão 2: caso fique confortável com o tempo de execução, tente inserir uma ou mais variáveis adicionais e veja se consegue aumentar a acurácia.

In [ ]:

```
```

3.1: Rode uma árvore para cada ccp_alpha, salvando cada árvore em uma lista

In [83]:

```
clfs = []

for ccp_alpha in ccp_alphas_sub:
    clf_alpha = DecisionTreeClassifier(
        random_state=100,
        min_samples_leaf=20,
        ccp_alpha=ccp_alpha
    )
    clf_alpha.fit(X_train_3, y_train)
    clfs.append(clf_alpha)

print("Quantidade de árvores treinadas:", len(clfs))
```

```
Quantidade de árvores treinadas: 47
```

In [ ]:

```
```

3.2: Calcule a acurácia de cada árvore na base de treinamento e de teste

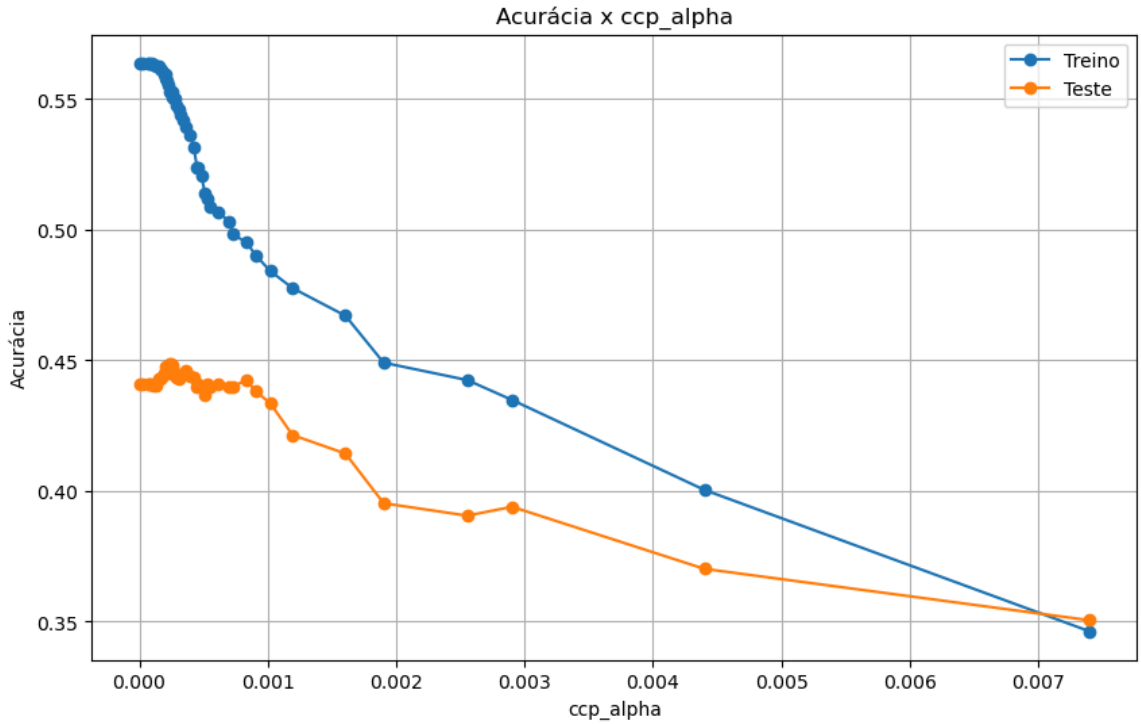
In [84]:
```
train_scores = []
test_scores = []

for clf_alpha in clfs:
    y_train_pred = clf_alpha.predict(X_train_3)
    y_test_pred = clf_alpha.predict(X_test_3)

    train_scores.append(accuracy_score(y_train, y_train_pred))
    test_scores.append(accuracy_score(y_test, y_test_pred))

print("Primeiras acurácias de treino:", train_scores[:5])
print("Primeiras acurácias de teste:", test_scores[:5])
```

```
Primeiras acurácias de treino: [0.5639281828073993, 0.5639281828073993, 0.
5639281828073993, 0.5639281828073993, 0.5636561479869423]
Primeiras acurácias de teste: [0.4407872412623006, 0.4407872412623006, 0.4
407872412623006, 0.4407872412623006, 0.4407872412623006]
```

In [ ]:

3.3: Monte um gráfico da acurácia em função do ccp_alpha na base de validação e na base de teste

In [86]:
```
plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas_sub, train_scores, marker='o', label='Treino')
plt.plot(ccp_alphas_sub, test_scores, marker='o', label='Teste')
plt.xlabel('ccp_alpha')
plt.ylabel('Acurácia')
plt.title('Acurácia x ccp_alpha')
plt.legend()
plt.grid(True)
plt.show()
```



In [ ]:

3.4: Selecione a melhor árvore como sendo aquela que possui melhor acurácia na base de teste

In [87]:
```
melhor_indice = np.argmax(test_scores)
melhor_alpha = ccp_alphas_sub[melhor_indice]
melhor_arvore = clfs[melhor_indice]
melhor_acuracia = test_scores[melhor_indice]

print("Melhor índice:", melhor_indice)
print("Melhor ccp_alpha:", melhor_alpha)
print("Melhor acurácia no teste:", melhor_acuracia)
```

```
Melhor índice: 15
Melhor ccp_alpha: 0.0002347367158915606
Melhor acurácia no teste: 0.44859178825924667
```

In [ ]:

3.5: Qual a melhor acurácia que se pode obter com estas três variáveis?

In [88]:
```
print(f"A melhor acurácia obtida com as três variáveis foi {melhor_acuracia:
```

```
A melhor acurácia obtida com as três variáveis foi 0.4486.
```

In [ ]:

```
Codigo inteiro junto
```

In [89]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Item 2 - features
features = pd.read_csv(
    './UCI HAR Dataset/features.txt',
    sep=r'\s+',
    header=None,
    names=['index', 'feature_name']
)
features_series = features['feature_name']
print("Quantidade de features:", len(features_series))
print(features_series.head())

# Item 3 - subject_train
subject_train = pd.read_csv(
    './UCI HAR Dataset/train/subject_train.txt',
    header=None,
    names=['subject']
)['subject']
print("Quantidade de registros em subject_train:", len(subject_train))
print(subject_train.head())

# Item 4 - X_train
X_train = pd.read_csv(
    './UCI HAR Dataset/train/X_train.txt',
    sep=r'\s+',
    header=None
)
X_train.columns = features_series
X_train['subject'] = subject_train.values
X_train['ordem'] = range(X_train.shape[0])
X_train.set_index(['ordem', 'subject'], inplace=True)
print("Dimensão de X_train:", X_train.shape)
print(X_train.head())

# Item 5 - subject_test e X_test
subject_test = pd.read_csv(
    './UCI HAR Dataset/test/subject_test.txt',
    header=None,
    names=['subject']
)['subject']

X_test = pd.read_csv(
    './UCI HAR Dataset/test/X_test.txt',
    sep=r'\s+',
    header=None
)
X_test.columns = features_series
X_test['subject'] = subject_test.values
X_test['ordem'] = range(X_test.shape[0])
X_test.set_index(['ordem', 'subject'], inplace=True)
print("Dimensão de X_test:", X_test.shape)
print(X_test.head())

# y_train e y_test
y_train = pd.read_csv(
    './UCI HAR Dataset/train/y_train.txt',
```

```python
    header=None,
    names=['activity']
)['activity']

y_test = pd.read_csv(
    './UCI HAR Dataset/test/y_test.txt',
    header=None,
    names=['activity']
)['activity']

print("y_train:")
print(y_train.head())
print("\ny_test:")
print(y_test.head())

# Item 6 - salvar CSV
X_train.to_csv('X_train.csv')
X_test.to_csv('X_test.csv')
y_train.to_csv('y_train.csv', index=False)
y_test.to_csv('y_test.csv', index=False)
print("Arquivos CSV salvos com sucesso.")

# Item 7 - selecionar 3 variáveis
colunas_selecionadas = [
    'tBodyAcc-mean()-X',
    'tBodyAcc-mean()-Y',
    'tBodyAcc-mean()-Z'
]

X_train_3 = X_train[colunas_selecionadas].copy()
X_test_3 = X_test[colunas_selecionadas].copy()

print("X_train_3:")
print(X_train_3.head())
print("\nX_test_3:")
print(X_test_3.head())

# Parte 2.1 - árvore inicial
clf = DecisionTreeClassifier(
    random_state=100,
    min_samples_leaf=20
)
clf.fit(X_train_3, y_train)

print("Árvore treinada com sucesso.")
print("Acurácia treino:", clf.score(X_train_3, y_train))
print("Acurácia teste:", clf.score(X_test_3, y_test))

# Parte 2.2 - ccp_alphas
path = clf.cost_complexity_pruning_path(X_train_3, y_train)
ccp_alphas = path.ccp_alphas
ccp_alphas = ccp_alphas[ccp_alphas >= 0]
ccp_alphas = np.unique(ccp_alphas)

print("Quantidade de ccp_alphas únicos e não negativos:", len(ccp_alphas))
print(ccp_alphas[:10])

# Parte 2.3 - 1 a cada 5
ccp_alphas_sub = ccp_alphas[::5]
print("Quantidade de ccp_alphas selecionados:", len(ccp_alphas_sub))
print(ccp_alphas_sub[:10])
```

```python
# Parte 3.1 - treinar várias árvores
clfs = []

for ccp_alpha in ccp_alphas_sub:
    clf_alpha = DecisionTreeClassifier(
        random_state=100,
        min_samples_leaf=20,
        ccp_alpha=ccp_alpha
    )
    clf_alpha.fit(X_train_3, y_train)
    clfs.append(clf_alpha)

print("Quantidade de árvores treinadas:", len(clfs))

# Parte 3.2 - acurácias
train_scores = []
test_scores = []

for clf_alpha in clfs:
    y_train_pred = clf_alpha.predict(X_train_3)
    y_test_pred = clf_alpha.predict(X_test_3)

    train_scores.append(accuracy_score(y_train, y_train_pred))
    test_scores.append(accuracy_score(y_test, y_test_pred))

print("Primeiras acurácias de treino:", train_scores[:5])
print("Primeiras acurácias de teste:", test_scores[:5])

# Parte 3.3 - gráfico
plt.figure(figsize=(10, 6))
plt.plot(ccp_alphas_sub, train_scores, marker='o', label='Treino')
plt.plot(ccp_alphas_sub, test_scores, marker='o', label='Teste')
plt.xlabel('ccp_alpha')
plt.ylabel('Acurácia')
plt.title('Acurácia x ccp_alpha')
plt.legend()
plt.grid(True)
plt.show()

# Parte 3.4 e 3.5 - melhor árvore
melhor_indice = np.argmax(test_scores)
melhor_alpha = ccp_alphas_sub[melhor_indice]
melhor_arvore = clfs[melhor_indice]
melhor_acuracia = test_scores[melhor_indice]

print("Melhor índice:", melhor_indice)
print("Melhor ccp_alpha:", melhor_alpha)
print("Melhor acurácia no teste:", melhor_acuracia)
print(f"A melhor acurácia obtida com as três variáveis foi {melhor_acuracia:
```

```
Quantidade de features: 561
0    tBodyAcc-mean()-X
1    tBodyAcc-mean()-Y
2    tBodyAcc-mean()-Z
3     tBodyAcc-std()-X
4     tBodyAcc-std()-Y
Name: feature_name, dtype: object
Quantidade de registros em subject_train: 7352
0    1
1    1
2    1
3    1
4    1
Name: subject, dtype: int64
Dimensão de X_train: (7352, 561)
feature_name   tBodyAcc-mean()-X  tBodyAcc-mean()-Y  tBodyAcc-mean()-Z
\
ordem subject
0     1                 0.288585          -0.020294          -0.132905
```

In [ ]:
```
%%time
```

In [ ]: